

UNIVERSIDAD PANAMERICANA

Facultad de ingeniería y Ciencias Aplicadas

Ingeniería en Sistemas y TIC

Análisis y Diseño de Sistemas



Análisis y Diseño de Sistemas

(Descripción de pruebas Proyecto Hipstagram)

Kevin Eleazar Azurdia Chamalé (000137188)

Juan Diego Wosbely Sac López (000149360)

Kendall Natanahel Abisaí Mux Chex (000150250)

Emerson Josué Vicente Martínez (000151734)

Jefry Fernando Berduo Pérez (000147971)

Chimaltenango, Chimaltenango, mayo de 2026

Tabla de contenido

DOCUMENTO DE DESCRIPCIÓN DE PRUEBAS.....	3
RESUMEN.....	3
PRUEBAS UNITARIAS E INTEGRACIÓN (JEST)	3
<i>Objetivo.....</i>	3
<i>Tecnologías utilizadas:</i>	3
<i>Configuración común (jest.config.js).....</i>	3
AUTH-SERVICE (44 TESTS)	4
<i>authController.test.ts</i>	4
<i>authMiddleware.test.ts.....</i>	4
<i>auth.integration.test.ts</i>	4
<i>e2e.integration.test.ts.....</i>	5
POST-SERVICE (26 TESTS)	5
<i>authMiddleware.test.ts.....</i>	5
<i>hashtagParser.test.ts.....</i>	5
<i>DoublyLinkedList.test.ts.....</i>	6
INTERACTIONS-SERVICE (10 TESTS)	6
<i>commentValidation.test.ts</i>	6
<i>authMiddleware.test.ts.....</i>	6
AUDIT-SERVICE (6 TESTS)	6
SEARCH-SERVICE (7 TESTS)	7
USER-SERVICE (12 TESTS)	7
MODERATION-SERVICE (14 TESTS)	8
RESUMEN TOTAL DE PRUEBAS UNITARIAS	9
CÓMO EJECUTAR LAS PRUEBAS	9
PRUEBAS DE CARGA (K6).....	9
OBJETIVO	9
TECNOLOGÍA	9
UMBRALES DE ACEPTACIÓN (DEFINIDOS EN TODOS LOS SCRIPTS)	10
PRUEBA 1 — FEED PRINCIPAL (GET /POSTS).....	10

<i>Escenario de carga:</i>	10
<i>Resultados obtenidos:</i>	10
PRUEBA 2 — BÚSQUEDA (GET /SEARCH/POSTS)	10
<i>Escenario de carga:</i>	10
<i>Resultados obtenidos:</i>	11
PRUEBA 3 — VOTACIÓN (POST /VOTO).....	11
<i>Escenario:</i>	11
<i>Resultados obtenidos:</i>	11
CÓMO REPRODUCIR LAS PRUEBAS DE CARGA.....	11
SEGURIDAD (OWASP TOP 10).....	12
PIPELINE CI/CD — STAGES DE CALIDAD	13
ESTRUCTURA DE ARCHIVOS DE PRUEBA.....	13
CONCLUSIONES	15

Documento de Descripción de Pruebas

Resumen

El proyecto Hipstagram implementa una arquitectura de 8 microservicios más 1 API Gateway. Para garantizar la calidad del software se ejecutaron cuatro tipos de pruebas:

Tipo de Prueba	Herramienta	Cobertura
Unitarias e Integración	Jest + ts-jest + supertest	119 tests, 7 microservicios
End-to-End (flujo completo)	Jest + supertest	9 pasos del ciclo de vida
Carga y Estrés	k6	3 endpoints críticos
Seguridad	OWASP Top 10 checklist	A01–A09 cubiertos

Pruebas Unitarias e Integración (Jest)

Objetivo

Verificar que cada microservicio funcione correctamente de forma aislada: lógica de negocio, validaciones, middleware de autenticación y respuestas HTTP.

Tecnologías utilizadas:

- Jest: Framework de pruebas (runner, mocks, assertions).
- ts-jest: compilación TypeScript en tiempo de prueba.
- supertest: cliente HTTP para probar endpoints sin levantar servidor.
- @types/jest: tipado TypeScript para Jest.

Configuración común (jest.config.js)

Todos los microservicios utilizan la siguiente configuración base:

```
module.exports = {  
  preset: 'ts-jest',  
  testEnvironment: 'node',  
  testMatch: ['**/*.test.ts'],  
  collectCoverage: true,  
  coverageDirectory: 'coverage',  
  coverageReporters: ['text', 'lcov']  
};
```

Los archivos de prueba se excluyen del build de producción mediante tsconfig.json:

```
{  
  "exclude": ["node_modules", "src/**/*.test.ts", "src/**/*.spec.ts"]  
}
```

auth-service (44 tests)

Ruta: backend/auth-service/src/

authController.test.ts

Prueba la lógica de negocio directamente (sin HTTP). Mockea la base de datos con jest.mock.

#	Caso de prueba	Resultado esperado
1	Login con credenciales correctas	Retorna accessToken y refreshToken
2	Login con contraseña incorrecta	Lanza error "Credenciales inválidas"
3	Login con cuenta deshabilitada	Lanza error "Cuenta deshabilitada"
4	Registro de nuevo usuario	Retorna usuario creado con id
5	Registro con email duplicado	Lanza error de conflicto
6	Refresh token válido	Retorna nuevo accessToken
7	Refresh token expirado/inválido	Lanza error 403
8	Reset de contraseña	Devuelve confirmación

authMiddleware.test.ts

Prueba el middleware JWT que protege rutas privadas.

#	Caso de prueba	Resultado esperado
1	Token válido en header Authorization	Llama a next() con req.user poblado
2	Sin header Authorization	Responde 401 Unauthorized
3	Token mal formado	Responde 403 Forbidden
4	Token expirado	Responde 403 con mensaje apropiado

auth.integration.test.ts

Prueba flujos HTTP reales con supertest.

#	Endpoint	Caso	Esperado
1	POST /login	Credenciales válidas	200 + tokens
2	POST /login	Contraseña incorrecta	401
3	POST /register	Datos completos	201 + usuario
4	POST /register	Email duplicado	409
5	POST /refresh	Token válido	200 + nuevo token

6	POST /reset-password	Usuario existente	200
---	----------------------	-------------------	-----

e2e.integration.test.ts

Flujo end-to-end completo del ciclo de vida de autenticación.

Paso	Acción	Verificación
1	POST /register	Usuario creado correctamente
2	POST /login	Tokens recibidos
3	GET /perfil (con token)	Datos del usuario
4	POST /refresh	Token renovado
5	Login con cuenta inactiva	Rechazado
6	Login contraseña incorrecta	Rechazado
7	Acceso sin token	401
8	Token malformado	403
9	POST /reset-password	Contraseña actualizada

Resultado: 44/44 tests pasando — Tiempo: 1.591s

post-service (26 tests)

Ruta: backend/post-service/src/

authMiddleware.test.ts

Prueba idéntica al patrón anterior — protección de rutas con JWT.

hashtagParser.test.ts

Prueba la función utilitaria que extrae hashtags de una descripción.

#	Entrada	Esperado
1	"Foto en #playa con #amigos"	["playa", "amigos"]
2	"Sin hashtags aquí"	[]
3	"#MAYÚSCULAS y #minúsculas"	Normaliza a minúsculas
4	"#repetido #repetido"	Deduplica
5	"" (cadena vacía)	[]

DoublyLinkedList.test.ts

Prueba la estructura de datos que actúa como buffer del feed en memoria.

#	Operación	Verificación
1	append()	Nodo agregado al final
2	prepend()	Nodo agregado al inicio
3	fromArray()	Lista construida desde arreglo
4	toArray()	Convierte lista a arreglo
5	traverse() adelante/atrás	Orden correcto en ambas direcciones
6	Lista vacía	No lanza errores

Resultado: 26/26 tests pasando — Tiempo: 0.477s

interactions-service (10 tests)

Ruta: backend/interactions-service/src/

commentValidation.test.ts

Prueba la validación de comentarios antes de insertarlos a la BD.

#	Caso	Resultado esperado
1	Comentario válido normal	{ valid: true }
2	Comentario vacío ""	Error: "No puede estar vacío"
3	Solo espacios en blanco	Error: inválido
4	Comentario > 128 caracteres	Error: "Demasiado largo"
5	Comentario con caracteres especiales	Permitido
6	Comentario con emojis	Permitido

authMiddleware.test.ts

Mismo patrón — verifica que /voto y /comentar requieren token.

Resultado: 10/10 tests pasando — Tiempo: 1.079s

audit-service (6 tests)

Ruta: backend/audit-service/src/auditController.test.ts

Prueba los endpoints del servicio que registra transacciones del sistema.

#	Endpoint	Caso	Esperado
1	POST /log	Payload completo	200 — log registrado
2	POST /log	Sin usuario_id	400 — campo requerido
3	POST /log	Sin accion	400 — campo requerido
4	POST /log	Payload mínimo válido	200
5	GET /health	Servicio activo	{ status: "ok" }
6	POST /log	Con actor_role y entity_type	200 — campos opcionales guardados

Resultado: 6/6 tests pasando

search-service (7 tests)

Ruta: backend/search-service/src/search.test.ts

Prueba el servicio de búsqueda de publicaciones por hashtag o descripción.

#	Endpoint	Caso	Esperado
1	GET /posts?q=playa	Búsqueda con coincidencias	200 + arreglo de posts
2	GET /posts?q=zzz999	Sin coincidencias	200 + arreglo vacío
3	GET /posts	Sin parámetro q	400 — parámetro requerido
4	GET /posts?q=a	Búsqueda de un carácter	200 (no falla)
5	GET /posts?q=playa&page=2	Paginación	200 + page=2
6	GET /health	Estado del servicio	{ status: "ok" }
7	GET /posts?q=MAYÚSCULA	Búsqueda case-insensitive	Encuentra resultados en minúsculas

Resultado: 7/7 tests pasando

user-service (12 tests)

Ruta: backend/user-service/src/user.test.ts

Prueba la gestión de perfiles y acciones administrativas sobre usuarios.

#	Endpoint	Caso	Esperado
1	GET /	Admin autenticado	200 + lista usuarios
2	GET /	Sin token	401
3	GET /	Token de USER (no admin)	403
4	GET /perfil/:id	ID existente	200 + datos perfil

5	GET /perfil/:id	ID inexistente	404
6	GET /perfil/:id	UUID mal formado	400 o 500 controlado
7	PUT /admin/status	Admin activa usuario	200 + confirmación
8	PUT /admin/status	Admin desactiva usuario	200 + confirmación
9	PUT /admin/status	Sin token	401
10	PUT /admin/role	Admin promueve a ADMIN	200
11	PUT /admin/role	Admin degrada a USER	200
12	PUT /admin/role	Rol inválido	400

Resultado: 12/12 tests pasando

moderation-service (14 tests)

Ruta: backend/moderation-service/src/moderation.test.ts

Prueba el servicio de moderación de contenido (palabras prohibidas + revisión de posts).

#	Endpoint	Caso	Esperado
1	POST /check	Texto limpio	{ clean: true }
2	POST /check	Texto con palabra prohibida	{ clean: false, details: [...] }
3	POST /check	Sin texto ni imagen	{ clean: true }
4	POST /check	Texto vacío	{ clean: true }
5	GET /words	Admin autenticado	200 + lista de palabras
6	GET /words	Sin token	401
7	POST /words	Agrega palabra nueva	200 + lista actualizada
8	POST /words	Palabra duplicada	Manejo sin error
9	POST /words	Sin token	401
10	DELETE /words/:word	Elimina palabra existente	200 + lista actualizada
11	DELETE /words/:word	Palabra no existe	404
12	DELETE /words/:word	Sin token	401
13	GET /health	Servicio activo	{ status: "ok" }
14	POST /check	Texto con mayúsculas en palabra bloqueada	{ clean: false } (case-insensitive)

Resultado: 14/14 tests pasando

Resumen total de pruebas unitarias

Microservicio	Archivo(s) de prueba	Tests	Estado
auth-service	authController, authMiddleware, auth.integration, e2e.integration	44	✓PASA
post-service	authMiddleware, hashtagParser, DoublyLinkedList	26	✓PASA
interactions-service	authMiddleware, commentValidation	10	✓PASA
audit-service	auditController	6	✓PASA
search-service	search	7	✓PASA
user-service	user	12	✓PASA
moderation-service	moderation	14	✓PASA
TOTAL	15 archivos	119	✓100% PASA

Cómo ejecutar las pruebas

Un solo microservicio

```
cd backend/auth-service
```

```
npm test
```

Con cobertura

```
npm run test:cov
```

Todos desde la raíz

```
for svc in auth-service post-service interactions-service audit-service search-service user-service  
moderation-service; do
```

```
  echo "=== $svc ===" && cd backend/$svc && npm test && cd ../..
```

```
done
```

Pruebas de Carga (k6)

Objetivo

Evaluar el comportamiento del sistema bajo condiciones de carga sostenida y picos de tráfico, verificando que los tiempos de respuesta sean aceptables y la tasa de error sea mínima.

Tecnología

k6 — herramienta open-source de pruebas de rendimiento basada en JavaScript.

Servidor evaluado: <https://3.88.254.85.nip.io> (instancia EC2 en producción)

Umbral de aceptación (definidos en todos los scripts)

```
thresholds: {  
  http_req_duration: ['p(95)<2000'], // 95% de requests < 2 segundos  
  http_req_failed: ['rate<0.05'], // Menos del 5% de errores  
  errors: ['rate<0.05']  
}
```

Prueba 1 — Feed principal (GET /posts)

Archivo: load-tests/feed.test.js

Escenario de carga:

Etapas	Duración	Usuarios
Rampa de subida	0s → 30s	0 → 50 VUs
Carga sostenida	30s → 60s	50 VUs
Pico máximo	60s → 90s	50 → 100 VUs
Enfriamiento	90s → 120s	100 → 0 VUs

Resultados obtenidos:

iterations.....: 5,304 43.87 req/s

http_req_duration....: avg=137ms p(90)=164ms p(95)=196ms max=1.63s

http_req_failed.....: 0.00% 0 de 5,304

vus_max.....: 100

Duración total.....: 2m 0.9s

Conclusión: Umbral superado — p(95) = 196ms (límite: 2000ms). Tasa de error: 0%.

Prueba 2 — Búsqueda (GET /search/posts)

Archivo: load-tests/search.test.js

Escenario de carga:

Etapas	Duración	Usuarios
--------	----------	----------

Rampa	0s → 30s	0 → 30 VUs
Sostenida	30s → 60s	30 VUs
Bajada	60s → 80s	30 → 0 VUs

Resultados obtenidos:

iterations.....: 1,646 20.38 req/s

http_req_duration....: avg=1.1s p(90)=1.11s p(95)=1.13s max=1.83s

http_req_failed.....: 0.00%

vus_max.....: 30

Duración total.....: 1m 20.8s

Conclusión: Umbral superado — p(95) = 1.13s (límite: 2000ms). Tasa de error: 0%.

Prueba 3 — Votación (POST /voto)

Archivo: load-tests/votos.test.js

Escenario:

Prueba de estrés directo sobre la lógica de votos (ON CONFLICT DO UPDATE).

Resultados obtenidos:

iterations.....: 4,359,959 72,666 iter/s

iteration_duration....: avg=205µs p(95)=384µs

vus_max.....: 20

Duración total.....: 1m 0s

Nota: Esta prueba validó directamente la lógica de la base de datos (sin red real). El volumen de 4.3 millones de iteraciones confirma que el patrón ON CONFLICT DO UPDATE es eficiente bajo alta concurrencia.

Conclusión: La lógica de votación es estable bajo carga extrema.

Cómo reproducir las pruebas de carga

Instalar k6 (macOS)

brew install k6

Ejecutar con servidor activo

k6 run load-tests/feed.test.js

k6 run load-tests/search.test.js

```
k6 run load-tests/votos.test.js
```

```
# Guardar resultados
```

```
k6 run --out json=docs/resultados-k6-feed.json load-tests/feed.test.js
```

Seguridad (OWASP Top 10)

Referencia completa: docs/owasp-checklist.md

Riesgo OWASP	Mitigación implementada	Estado
A01 — Broken Access Control	JWT + RBAC (USER/ADMIN), verificarToken en rutas privadas	✓
A02 — Cryptographic Failures	bcrypt para contraseñas, HTTPS en producción (nip.io + cert)	✓
A03 — Injection	Queries parametrizadas (\$1, \$2...) en todos los microservicios, nunca concatenación	✓
A04 — Insecure Design	Microservicios con responsabilidad única, sin lógica compartida por referencia	✓
A05 — Security Misconfiguration	x-powered-by deshabilitado, CORS restringido por ALLOWED_ORIGIN	✓
A06 — Vulnerable Components	npm audit --audit-level=high en Stage 3 del pipeline Jenkins	✓
A07 — Auth Failures	Tokens con expiración (8h access, 7d refresh), refresh token rotation	✓
A08 — Integrity Failures	Imágenes solo en S3 con URL firmada, no en filesystem expuesto	✓
A09 — Logging & Monitoring	Audit service registra todas las transacciones con IP, rol, resultado	✓

Pipeline CI/CD — Stages de Calidad

El pipeline Jenkins ejecuta las pruebas automáticamente en cada push al repositorio.

Stage	Descripción
1. Checkout	Clona el repositorio
2. Build Docker Images	Construye todas las imágenes
3. Lint y Seguridad	npm audit --audit-level=high por servicio
4. Pruebas Unitarias	npm test en auth, post, interactions, audit, search, user, moderation
5. SonarQube Analysis	Análisis de calidad de código y cobertura
6. Quality Gate	Bloquea el pipeline si SonarQube falla
7. Push a ECR	Sube imágenes a AWS Elastic Container Registry
8. Deploy a EC2	Actualiza los contenedores en producción
9. Health Check	Verifica que todos los servicios respondan /health

Estructura de archivos de prueba

proyecto-hipstagram/

```
|— backend/
| |— auth-service/src/
| | |— controllers/authController.test.ts
| | |— authMiddleware.test.ts
| | |— auth.integration.test.ts
| | |— e2e.integration.test.ts
| |— post-service/src/
| | |— authMiddleware.test.ts
| | |— helpers/hashtagParser.test.ts
| | |— structures/DoublyLinkedList.test.ts
| |— interactions-service/src/
| | |— authMiddleware.test.ts
| | |— helpers/commentValidation.test.ts
| |— audit-service/src/
| | |— auditController.test.ts
| |— search-service/src/
| | |— search.test.ts
| |— user-service/src/
```

```
| | └─ user.test.ts
| └─ moderation-service/src/
|   └─ moderation.test.ts
└─ load-tests/
    | └─ feed.test.js
    | └─ search.test.js
    | └─ votos.test.js
    └─ docs/
        └─ descripcion-pruebas.md ← este documento
        └─ evidencias-pruebas.md
        └─ owasp-checklist.md
        └─ resumen-k6-feed.txt
        └─ resumen-k6-search.txt
        └─ resumen-k6-votos.txt
```

Conclusiones

1. Cobertura unitaria completa: 119 pruebas distribuidas en 7 microservicios, todas pasando al 100%.
2. Flujo e2e cubierto: El ciclo completo de autenticación (registro → login → uso de token → refresh → reset) fue probado con 9 pasos encadenados.
3. Rendimiento bajo carga: El sistema soportó 100 usuarios concurrentes en el feed con tiempos de respuesta de $p(95)=196\text{ms}$, muy por debajo del umbral de 2000ms.
4. Seguridad verificada: Se implementaron y verificaron mitigaciones para las 9 categorías OWASP relevantes al proyecto.
5. Automatización: Las pruebas unitarias se ejecutan automáticamente en el pipeline Jenkins (Stage 4) en cada push, garantizando regresión continua.